```
$ docker-compose up -d
Starting myapp_web_46768de21d89 ... done
```

Note, however, that it may take some time for Rails to launch and the application to become available.

Problem Starting Rails?



If you encounter an error on starting the server that says something like, "A server is already running," you've run into a bug with Compose. For now, simply delete the tmp/pids/server.pid on your local machine. See Chapter 9 for a better solution.

Mounting a Local Volume

Before we leave our docker-compose.yml, let's make a small addition.

We've already seen how to mount a local directory inside a container with docker run by using the -v option—we did this in Generating a New Rails App Without Ruby Installed, on page 10, so that the Rails project files generated inside the container would be available on our local machine.

A mounted local volume represents some filesystem that's shared between your local machine and the container. Files in the mounted volume are synced both ways between your local filesystem and the container. Because of this, a local volume mount can allow us to develop locally and have the Rails server running in the container automatically pick up the file changes without restarting—just like we're used to.

We are going to set up this locally mounted volume using Compose rather than using docker run this time. In our docker run command, we used the option -v $PWD:/usr/src/app to mount our current, local directory inside the Rails container at /usr/src/app. We can achieve the same with Compose by adding the following to our docker-compose.yml:

```
version: '3'

services:

  web:
    build: .
    ports:
      - "3000:3000"
    volumes:
      - .:/usr/src/app
```

Here we specify our volume mapping under the volumes property we've added for the web service. Although the volume mapping is very similar to the docker run option we used, there is one slight difference. We were able to refer to the